CLIENTOPS CRM

ClientOps

Multi-tenant CRM SaaS — Laravel 12 + React 19

Architektura i Model Danych

Kompletna dokumentacja techniczna: warstwy danych, diagram ERD, pełne schematy wszystkich tabel, relacje, przepływy biznesowe i struktura UI. Główne źródło prawdy dla developera.

WERSJA	DATA	TABELE	SEKCJE
2.0.0	Marzec 2026	27	13

Spis treści

Dokument zawiera 13 sekcji — od warstw danych po schematy tabel i przepływy biznesowe

1 Podział systemu na warstwy — 27 tabel

2 ERD — diagram relacji (Mermaid)

3 Schematy tabel — Warstwa 1: Tożsamość

4 Schematy tabel — Warstwa 2: Sprzedaż

5 Schematy tabel — Warstwa 3: Projekty i dokumenty

6 Schematy tabel — Warstwa 3: SLA, tickety, Kanban

7 Schematy tabel — Warstwa 3: Inbox, audit log, SEO

8 Flow sprzedaży — od leadu do projektu

9 Kanban — statusy, kolumny, ujednolicony model

10 Tickets vs SLA Incidents — dwie sfery

11 Inbox i wiadomości wewnętrzne — flow

12 Audyty i analizy SEO — pipeline

13 Struktura menu — panel admina i klienta

1

ARCHITECTURE

Podział systemu na warstwy — 27 tabel

System podzielony na trzy warstwy logiczne. Każda warstwa obsługuje inny obszar biznesowy i zależy od warstwy poprzedniej — nie można mieć projektu bez klienta, klienta bez kontaktu.

Warstwa 1 — Tożsamość (3 tabele)

`users` `clients` `contacts`

Warstwa 2 — Sprzedaż (3 tabele)

`leads` `lead_activities` `offers`

Warstwa 3 — Realizacja (21 tabel)

`projects` `documents` `document_versions` `kanban_boards` `kanban_columns`
`kanban_cards` `sla_policies` `tickets` `ticket_comments` `sla_incidents`
`notifications` `notification_preferences` `conversations` `conversation_participants`
`messages` `mentions` `activity_log` `audits` `seo_analyses` `seo_issues`
`seo_keyword_ranks`

Kluczowa zasada architektury

Lead → Klient → Projekt to trzy osobne stany relacji biznesowej. Lead staje się klientem przez eksplicytną konwersję (event + listener). Projekt jest tworzony dla klienta — nigdy bezpośrednio dla leadu. Nie skracaj tej ścieżki.

Multi-tenancy — single database + Global Scopes

Każda tabela w warstwie 2 i 3 zawiera `client_id`. Middleware identyfikuje tenant po subdomenie i wstrzykuje go do kontenera. Trait `BelongsToTenant` dodaje Global Scope automatycznie — zapytania są zawsze filtrowane bez potrzeby jawnego `where('client_id', ...)`.

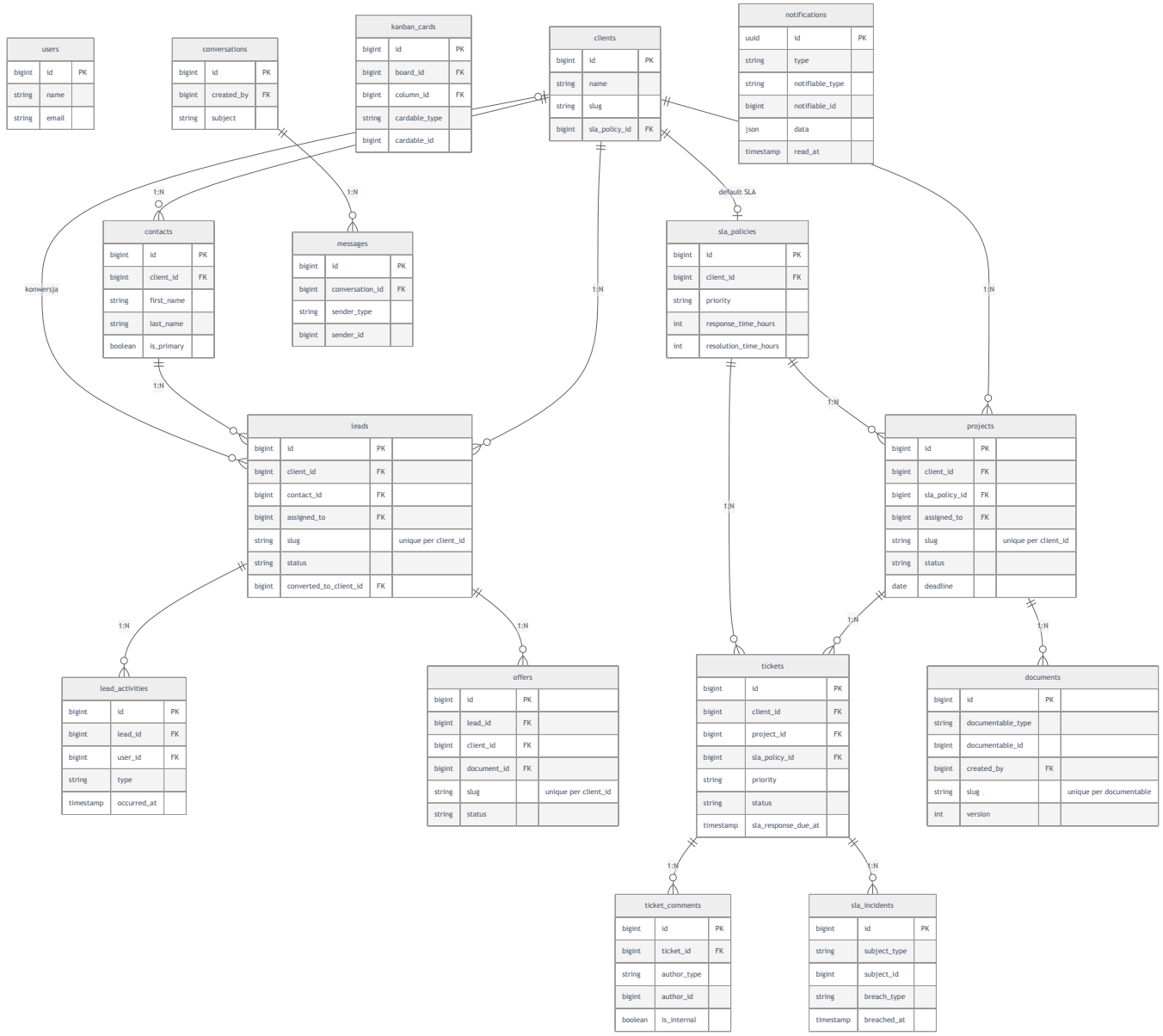
```php
// Middleware — identyfikacja tenanta po subdomenie
$slug   = explode('.', $request->getHost())[0]; // "acme" z acme.clientops.pl
$tenant = Client::where('slug', $slug)->firstOrFail();
app()->instance('tenant', $tenant);

// Trait BelongsToTenant — dodaje Global Scope + auto-fill client_id
static::addGlobalScope(new TenantScope);
static::creating(fn($model) => $model->client_id = app('tenant')->id);
```

2

DATA MODEL

ERD — diagram relacji

Diagram pokazuje klucze główne (PK), klucze obce (FK) i typy relacji. Tabele polymorphic (`kanban_cards` , `activity_log` , `mentions`) obsługują wiele typów podmiotów przez `*_type` + `*_id` .



3

SCHEMAS

Schematy tabel — Warstwa 1: Tożsamość

users

```
users
  id                bigint PK auto_increment
  name              varchar(255)
  email             varchar(255) unique
  email_verified_at timestamp nullable
  password          varchar(255)                -- bcrypt hash
  remember_token    varchar(100) nullable
  avatar_path       varchar(255) nullable        -- Spatie Media Library lub null
  phone             varchar(20) nullable
  is_active         boolean default true
  last_login_at     timestamp nullable
  created_at        timestamp
  updated_at        timestamp

  -- Role zarządzane przez Spatie Permission (tabela model_has_roles)
  -- Dostępne role: super-admin | admin | manager | employee | client-admin | client-user
```

clients

```
clients
  id                    bigint PK auto_increment
  name                  varchar(255)                -- nazwa firmy
  slug                  varchar(100) unique         -- subdomena: acme → acme.clientops.pl
  nip                   varchar(20) nullable
  address               varchar(255) nullable
  city                  varchar(100) nullable
  postal_code           varchar(10) nullable
  country               varchar(2) default 'PL'
  website               varchar(255) nullable
  status                enum('active','inactive','archived') default 'active'
  sla_policy_id         bigint nullable FK → sla_policies.id  -- domyślna polityka SLA
  converted_from_lead_id bigint nullable FK → leads.id       -- skąd pochodzi
  notes                 text nullable
  created_by            bigint FK → users.id
  created_at            timestamp
  updated_at            timestamp
```

contacts

```
contacts
  id              bigint PK auto_increment
  client_id       bigint FK → clients.id
  first_name      varchar(100)
  last_name       varchar(100)
  email           varchar(255) nullable
  phone           varchar(20) nullable
  position        varchar(100) nullable       -- stanowisko w firmie
  is_primary      boolean default false        -- główny kontakt firmy
  notes           text nullable
  created_at      timestamp
  updated_at      timestamp

  -- Index: (client_id), (email)
```

Uwaga: users vs contacts

`users` to konta w systemie (login, role, hasło). `contacts` to osoby kontaktowe w firmach klientów — mogą, ale nie muszą mieć konta użytkownika. Klient loguje się jako `user` z rolą `client-admin` lub `client-user`, powiązanym z `client_id`.

4

SCHEMAS

Schematy tabel — Warstwa 2: Sprzedaż

leads

```
leads
  id                      bigint PK auto_increment
  client_id               bigint nullable FK → clients.id   -- null przed konwersją
  contact_id              bigint nullable FK → contacts.id  -- osoba kontaktowa
  assigned_to             bigint nullable FK → users.id
  title                   varchar(255)                      -- np. "SEO dla firmy ABC"
  company_name            varchar(255) nullable             -- przed konwersją do clients
  slug                    varchar(100)                      -- URL identifier, unique per client_id
  status                  enum('new','contacted','no_response',
                               'in_discussion','offer_sent','won','lost')
                          default 'new'
  source                  enum('manual','import','form','referral') default 'manual'
  value                   decimal(12,2) nullable            -- szacowana wartość
  expected_close_date     date nullable
  notes                   text nullable
  converted_at            timestamp nullable                -- kiedy status → won
  converted_to_client_id  bigint nullable FK → clients.id  -- powstały klient
  created_by              bigint FK → users.id
  created_at              timestamp
  updated_at              timestamp

  -- Index: (status), (assigned_to), (client_id)
  -- UNIQUE: (slug, client_id)
```

lead_activities

```
lead_activities
  id            bigint PK auto_increment
  lead_id       bigint FK → leads.id
  user_id       bigint FK → users.id              -- kto wykonał aktywność
  type          enum('call','email','meeting','note','status_change')
  description   text nullable
  occurred_at   timestamp                         -- kiedy miała miejsce (nie created_at!)
  metadata      json nullable
                  -- dla status_change: { "from": "new", "to": "contacted" }
                  -- dla call: { "duration_minutes": 15 }
                  -- dla meeting: { "location": "...", "attendees": [...] }
  created_at    timestamp
```

offers

```
offers
  id                      bigint PK auto_increment
  lead_id                 bigint FK → leads.id
  client_id               bigint nullable FK → clients.id  -- wypełniane przy konwersji
  title                   varchar(255)
  slug                    varchar(100)                      -- URL identifier, unique per client_id
  status                  enum('draft','sent','accepted','rejected','expired')
                          default 'draft'
  value                   decimal(12,2) nullable
  valid_until             date nullable
  document_id             bigint nullable FK → documents.id -- opcjonalny TipTap
  notes                   text nullable
  accepted_at             timestamp nullable
  rejected_at             timestamp nullable
  converted_to_client_at  timestamp nullable
  created_by              bigint FK → users.id
  created_at              timestamp
  updated_at              timestamp

  -- UNIQUE: (slug, client_id)
```

5

SCHEMAS

Schematy tabel — Warstwa 3: Projekty i dokumenty

projects

```
projects
  id              bigint PK auto_increment
  client_id       bigint FK → clients.id
  name            varchar(255)
  slug            varchar(100)                  -- URL identifier, unique per client_id
  description     text nullable
  status          enum('planning','active','on_hold','completed','cancelled')
                  default 'planning'
  sla_policy_id   bigint nullable FK → sla_policies.id
  assigned_to     bigint nullable FK → users.id    -- główny opiekun projektu
  deadline        date nullable
  started_at      timestamp nullable
  completed_at    timestamp nullable
  created_by      bigint FK → users.id
  created_at      timestamp
  updated_at      timestamp

  -- UNIQUE: (slug, client_id)
```

documents

```
documents
  id                bigint PK auto_increment
  documentable_type polymorphic varchar(255)  -- "Project" | "Client" | "Ticket"
  documentable_id   bigint
  title             varchar(255)
  slug              varchar(100)                  -- URL identifier, unique per documentable
  content_json      longtext nullable              -- TipTap JSON format
  content_html      longtext nullable              -- wygenerowany HTML (cache)
  pdf_path          varchar(255) nullable          -- ścieżka do S3/MinIO
  version           int default 1
  is_published      boolean default false          -- czy widoczny dla klienta
  created_by        bigint FK → users.id
  created_at        timestamp
  updated_at        timestamp

  -- Index: (documentable_type, documentable_id)
  -- UNIQUE: (slug, documentable_type, documentable_id)
```

document_versions

```
document_versions
  id              bigint PK auto_increment
  document_id     bigint FK → documents.id
  version         int                     -- snapshot numeru wersji
  content_json    longtext                -- pełny snapshot TipTap JSON
  title           varchar(255)            -- tytuł w momencie zapisu
  created_by      bigint FK → users.id
  created_at      timestamp               -- tylko created_at, bez updated_at (immutable)
```

Wersjonowanie dokumentów

Przy każdym zapisie dokumentu: (1) inkrementuj `documents.version`, (2) utwórz snapshot w `document_versions`. Stare wersje nigdy nie są nadpisywane. PDF generuje się asynchronicznie z bieżącej wersji (Browsershot + kolejka).

sla_policies

```
sla_policies
  id                    bigint PK auto_increment
  client_id             bigint nullable FK → clients.id  -- null = globalny default
  name                  varchar(255)                     -- np. "Standard", "Premium 24/7"
  priority              enum('low','medium','high','critical')
  response_time_hours   int                              -- max czas do pierwszej odpowiedzi
  resolution_time_hours int                              -- max czas do zamknięcia
  business_hours_only   boolean default true             -- czy liczyć tylko godziny robocze
  created_at            timestamp
  updated_at            timestamp
```

6

SCHEMAS

Schematy tabel — Warstwa 3: SLA, tickety, Kanban

tickets

```
tickets
  id                    bigint PK auto_increment
  client_id             bigint FK → clients.id
  project_id            bigint FK → projects.id
  created_by            bigint                          -- user_id (client-user lub admin)
  created_by_type       varchar(50)                     -- "User" | "Client"
  title                 varchar(255)
  description           text
  priority              enum('low','medium','high','critical') default 'medium'
  status                enum('new','open','in_progress','waiting','resolved','closed')
                        default 'new'
  sla_policy_id         bigint nullable FK → sla_policies.id
  sla_response_due_at   timestamp nullable              -- obliczane przy tworzeniu
  sla_resolution_due_at timestamp nullable
  first_responded_at    timestamp nullable              -- kiedy admin pierwszy raz odpowiedział
  resolved_at           timestamp nullable
  closed_at             timestamp nullable
  kanban_card_id        bigint nullable FK → kanban_cards.id
  created_at            timestamp
  updated_at            timestamp
```

ticket_comments

```
ticket_comments
  id              bigint PK auto_increment
  ticket_id       bigint FK → tickets.id
  author_id       bigint                        -- polymorphic: user lub client user
  author_type     polymorphic varchar(50)       -- "User" | "Client"
  is_internal     boolean default false         -- notatki wewnętrzne niewidoczne dla klienta
  content         text
  created_at      timestamp
  updated_at      timestamp
```

sla_incidents

```
sla_incidents
  id                bigint PK auto_increment
  subject_type      polymorphic varchar(50)   -- "Ticket" | "Project"
  subject_id        bigint
  breach_type       enum('response','resolution','deadline')
  breached_at       timestamp                    -- kiedy SLA zostało przekroczone
  detected_at       timestamp                    -- kiedy job to wykrył
  notified          boolean default false         -- czy powiadomienie wysłane
  acknowledged_by   bigint nullable FK → users.id
  acknowledged_at   timestamp nullable
  resolved_at       timestamp nullable
  created_at        timestamp
```

kanban_boards / kanban_columns / kanban_cards

```
kanban_boards
  id          bigint PK
  name        varchar(255)
  board_type  enum('leads',
              'tickets',
              'project_tasks')
  created_at  timestamp

kanban_columns
  id          bigint PK
  board_id    bigint FK
  name        varchar(255)
  position    int
  color       varchar(7) nullable
  created_at  timestamp
```

```
kanban_cards
  id            bigint PK
  board_id      bigint FK
  column_id     bigint FK
  position      int
    -- polymorphic subject:
  cardable_type poly varchar
    -- "Lead"|"Ticket"
    -- |"ProjectTask"
  cardable_id   bigint
  assigned_to   bigint nullable FK
  due_at        timestamp nullable
  created_at    timestamp
  updated_at    timestamp
```

7

SCHEMAS

Schematy tabel — Warstwa 3: Inbox, audit log, SEO

notifications / notification_preferences

```
notifications
  id                uuid PK
  type              varchar(255)
    -- sla_breach | mention
    -- message | task_assigned
    -- comment_reply | pdf_ready
    -- new_ticket | audit_complete
  notifiable_type   poly  varchar
    -- "User" | "Client"
  notifiable_id     bigint
  data              json
    -- { body, sender_id,
    --   link, subject_type,
    --   subject_id }
  read_at           timestamp nullable
  created_at        timestamp
```

```
notification_preferences
  id                bigint PK
  user_id           bigint FK
  notification_type varchar
  channels          json
    -- ['mail','sms',
    --  'database',
    --  'broadcast']
  quiet_hours_from time nullable
  quiet_hours_to   time nullable
  quiet_hours_timezone
                    varchar default
                    'Europe/Warsaw'
  created_at        timestamp
  updated_at        timestamp
```

conversations / conversation_participants / messages / mentions

```
conversations
  id          bigint PK
  subject     varchar nullable
  created_by  bigint FK → users
  created_at  timestamp

conversation_participants
  id                bigint PK
  conversation_id   bigint FK
  participable_type poly
      -- "User"|"Client"
  participable_id   bigint
  last_read_at      timestamp nullable
```

```
messages
  id                bigint PK
  conversation_id bigint FK
  sender_type       poly  varchar
    -- "User" | "Client"
  sender_id         bigint
  body              text
  created_at        timestamp

mentions
  id                  bigint PK
  mentionable_type    poly  varchar
    -- "Message"
    -- "TicketComment"
  mentionable_id      bigint
  mentioned_type      varchar
  mentioned_id        bigint
  created_at          timestamp
```

activity_log (Spatie Activity Log)

```
activity_log
  id              bigint PK auto_increment
  log_name        varchar(255) nullable   -- "auth" | "crm" | "documents" | "tickets" | "system"
  description     text                    -- "created" | "updated" | "deleted" | "login" | ...
  subject_type    polymorphic varchar     -- "Lead" | "Client" | "Ticket" | "User" | ...
  subject_id      bigint nullable
  causer_type     polymorphic varchar     -- "User" (kto wykonał akcję)
  causer_id       bigint nullable
  properties      json nullable
                     -- { "old": { "status": "new" }, "attributes": { "status": "contacted" } }
  created_at      timestamp
```

slug_redirects

```
slug_redirects
  id               bigint PK auto_increment
  sluggable_type   varchar(100)        -- "Client" | "Project" | "Lead" | "Document" | ...
  sluggable_id     bigint
  old_slug         varchar(100)        -- poprzedni slug przed zmianą
  changed_by       bigint FK → users.id
  created_at       timestamp

  -- Index: (sluggable_type, old_slug)  ← używany przez middleware redirect
```

Polityka slug w systemie

Slug generowany automatycznie przy tworzeniu rekordu (`spatie/laravel-sluggable` , `onUpdate = false`). Zmiana tytułu/nazwy nie aktualizuje slugu. Ręczna zmiana slugu przez admina zapisuje stary slug do `slug_redirects` — middleware zwraca `301 Redirect` na aktualny URL. Kolizja slugu w tym samym scope → auto-suffix: `umowa` → `umowa-2` → `umowa-3` .

seo_analyses / seo_issues / seo_keyword_ranks

```
seo_analyses
  id             bigint PK
  project_id     bigint FK
  audit_id       bigint nullable FK
  status         enum('pending',
                 'running',
                 'completed',
                 'failed')
  score          int nullable
  keywords_count int default 0
  issues_count   int default 0
  gsc_connected  boolean default false
  ga4_connected  boolean default false
  completed_at   timestamp nullable
  created_at     timestamp

seo_issues
  id               bigint PK
  seo_analysis_id  bigint FK
  type             varchar(100)
  severity         enum('critical',
                   'warning','info')
  url              varchar(500)
  description      text
  fixed_at         timestamp nullable
```

```
seo_keyword_ranks
    -- TimescaleDB hypertable
    -- partitioned by: time
  id               bigint PK
  seo_analysis_id  bigint FK
  keyword          varchar(255)
  position         int
  clicks           int default 0
  impressions      int default 0
  ctr              decimal(5,4)
  url              varchar(500)
  device           enum('desktop',
                   'mobile','tablet')
  time             timestamp
                     -- PRIMARY TIME COLUMN
                     -- do continuous aggregates

audits
  id             bigint PK
  project_id     bigint FK
  type           enum('seo',
                 'technical',
                 'ux','security',
                 'full')
  status         enum('pending',
                 'running',
                 'completed',
                 'failed')
  overall_score  int nullable
  started_at     timestamp nullable
  completed_at   timestamp nullable
  created_by     bigint FK
```

8

PROCESS

Flow sprzedaży — od leadu do projektu

Kompletny proces od pierwszego kontaktu do aktywnej realizacji. Każdy krok to osobny byt — nie skróty, nie aliasy.

Główny flow

Kontakt / firma → Lead (nowy) → Lead (w rozmowie) → Oferta → Klient → Projekt

Projekt → Dokumenty + Analizy SEO + Audyty + SLA monitoring

Krok po kroku — co dzieje się w bazie

#	Akcja	Zapisy w bazie	Side effects
1	Admin tworzy lead	`leads` (status: new) opcjonalnie: `contacts`	Karta Kanban w kolumnie "Nowy" `activity_log` (created)
2	Kontakt z leadem	`lead_activities` (call/email/meeting)	Zmiana statusu → karta Kanban przesuwa się `activity_log` (status_change)
3	Wysyłka oferty	`offers` (status: sent) opcjonalnie: `documents` + `document_versions`	Lead → status: offer_sent `notifications` dla przypisanego admina
4	Oferta zaakceptowana	`offers.status` → accepted `offers.accepted_at` = now()	`LeadOfferAccepted` event fired
5	Konwersja lead → klient	Nowy rekord `clients` `leads.status` → won `leads.converted_at` = now() `leads.converted_to_client_id` = new_client.id	Kopiowanie `contacts` Karta Kanban → kolumna "Wygrany" `activity_log` (converted_to_client)
6	Tworzenie projektu	Nowy rekord `projects` z client_id Przypisanie `sla_policy_id`	Subdomena klienta aktywna `activity_log` (created)

Event-driven — konwersja leadu

```php
// Event: LeadWonEvent::class (fired gdy leads.status → 'won')
class LeadWonListener
{
    public function handle(LeadWonEvent $event): void
    {
        $lead   = $event->lead;
        $client = Client::create([
            'name'                  => $lead->company_name,
            'slug'                  => Str::slug($lead->company_name),
            'converted_from_lead_id'=> $lead->id,
            'sla_policy_id'         => SlaPolicy::default()->id,
        ]);

        $lead->contacts->each(fn($c) => $c->update(['client_id' => $client->id]));
        $lead->update(['converted_to_client_id' => $client->id, 'converted_at' => now()]);

        activity('crm')
            ->on($lead)
            ->log('converted_to_client');
    }
}
```

9

UI / PROCESS

Kanban — statusy, kolumny, ujednolicony model

Statusy leadu — PHP Enum

```
enum LeadStatus: string
{
    case New          = 'new';
    case Contacted    = 'contacted';
    case NoResponse   = 'no_response';
    case InDiscussion = 'in_discussion';
    case OfferSent    = 'offer_sent';
    case Won          = 'won';     // terminal → wyzwala konwersję do klienta
    case Lost         = 'lost';    // terminal → karta znika z głównego widoku
}
```

Ujednolicony model — jeden Kanban dla trzech typów

Ten sam model `kanban_cards` obsługuje leady, tickety i zadania projektowe przez polymorphic relations. Zmiana statusu leadu automatycznie przesuwa kartę przez event-listener.

```
// Event → Listener: synchronizacja statusu z pozycją karty
class MoveKanbanCardListener
{
    public function handle(LeadStatusChangedEvent $event): void
    {
        $column = KanbanColumn::where('board_id', $event->boardId)
                              ->where('slug', $event->newStatus)
                              ->first();

        $event->lead->kanbanCard->update(['column_id' => $column->id]);
    }
}
```

cardable_type	Źródło statusu	Kolumny
`"Lead"`	`leads.status` (enum)	new → contacted → no_response → in_discussion → offer_sent → won → lost

cardable_type	Źródło statusu	Kolumny
`"Ticket"`	`tickets.status` (enum)	new → open → in_progress → waiting → resolved → closed
`"ProjectTask"`	ręczne drag & drop	todo → in_progress → review → done

10

ARCHITECTURE

Tickets vs SLA Incidents — dwie sfery

tickets — sfera ludzka

Twórca	klient (ręcznie)
Wyzwalacz	człowiek klika "zgłoś"
Cel	obsługa zgłoszenia
Widoczny	klient + admin
Komentarze	tak (+ wewnętrzne)
Kanban	tak → karta admina

sla_incidents — sfera systemowa

Twórca	system (automatycznie)
Wyzwalacz	upływ czasu / brak akcji
Cel	alert o naruszeniu SLA
Widoczny	admin / manager
Komentarze	nie
Kanban	zmiana koloru karty

Warunki wyzwalające sla_incident

breach_type	subject_type	Warunek SQL
`response`	Ticket	`sla_response_due_at < NOW() AND first_responded_at IS NULL`
`resolution`	Ticket	`sla_resolution_due_at < NOW() AND resolved_at IS NULL`
`deadline`	Project	`deadline < NOW() AND status ≠ 'completed'`

Scheduled job — CheckSlaBreachesJob (co 5 min)

```
Schedule::job(CheckSlaBreachesJob::class)→everyFiveMinutes();

CheckSlaBreachesJob skanuje:
├── tickets: sla_response_due_at < now() AND first_responded_at IS NULL
|     → INSERT sla_incidents (breach_type: response)
|     → Notification do admina (mail + SMS jeśli priority: critical)
|     → Kanban karta → kolor czerwony
|
├── tickets: sla_resolution_due_at < now() AND resolved_at IS NULL
|     → INSERT sla_incidents (breach_type: resolution)
|     → Notification do managera
|
└── projects: deadline < now() AND status ≠ 'completed'
      → INSERT sla_incidents (breach_type: deadline)
      → Notification in-app + email
```

Jeden ticket może mieć wiele sla_incidents

> Np. najpierw `response_breach` po 4h bez odpowiedzi, potem `resolution_breach` po 24h. Każdy incydent to osobny rekord z własnym `detected_at` i niezależnym procesem `acknowledged`.

MODULE

11 Inbox i wiadomości wewnętrzne — flow

Dwa strumienie zdarzeń trafiają do jednego inboxu. Różnią się źródłem — ale dostarczane są tym samym mechanizmem: tabela `notifications` + broadcast przez Reverb.

Flow — każde zdarzenie wymagające uwagi

```
┌──────────────────────────────────────────────────────────┐
│                      ŹRÓDŁA ZDARZEŃ                      │
└──────────────────────────────────────────────────────────┘

  System automatyczny          Człowiek (admin / klient)
  ___________________          _________________________

  SLA breach                   Admin pisze do admina
  @mention w komentarzu        Admin pisze do klienta
  Task assigned                Klient pisze do admina
  Ticket zgłoszony             @mention w wiadomości
  PDF gotowy
  Audyt ukończony
        │                              │
        └──────────────┬───────────────┘
                       ▼
           INSERT INTO notifications
           (type, notifiable_type/id, data)
                       |
                       ▼
           broadcast('private-user.' . $userId)
             via Laravel Reverb
                       |
                       ▼
           Echo.private('user.' + id)
             .listen('.NotificationCreated', ...)
                       |
                       ▼
           Zustand store: unread_count++
           Bell icon w navbar się podświetla
```

Po kliknięciu w powiadomienie

type	data.link prowadzi do	Email?
`sla_breach`	Widok naruszenia SLA	✅ + SMS (critical)
`new_ticket`	Ticket w panelu admina	✅
`mention`	Komentarz w kontekście (ticket / projekt)	wg preferencji

type	data.link prowadzi do	Email?
`task_assigned`	Kanban karta / zadanie	wg preferencji
`message`	Wątek konwersacji w inboxie	wg preferencji
`pdf_ready`	Link do pobrania PDF	—
`audit_complete`	Raport audytu SEO	✅

Flow wiadomości — zapis + broadcast

```
// Admin A pisze do Admin B
$message = Message::create([
    'conversation_id' => $conv->id,
    'sender_type'     => 'User',
    'sender_id'       => auth()->id(),
    'body'            => $request->body,
]);

// Powiadom wszystkich uczestników (poza nadawcą)
$conv->participants
    ->reject(fn($p) => $p->participable_id === auth()->id())
    ->each(function($participant) use ($message) {
        $participant->participable->notify(
            new NewMessageNotification($message)
        );
    });

// NewMessageNotification via('database', 'broadcast') + opcjonalnie 'mail'
```

12

MODULE

Audyty i analizy SEO — pipeline

Analiza SEO może być bytem samodzielnym (`audit_id = null`) lub składową audytu. Pipeline kolejkowy uruchamia się asynchronicznie przez Laravel Horizon.

Dwa tryby użycia seo_analyses

Analiza samodzielna

audit_id	NULL
Widoczna w	"Analizy SEO" projektu
Typ	miesięczny monitoring

Składowa audytu

audit_id	= audits.id
Widoczna w	raport audytu + agregat
Typ	audyt kwartalny

Pipeline kolejkowy (Horizon)

```
AuditRequestedEvent
  |
  ├── CrawlWebsiteJob        (queue: crawlers)
  |     → seo_audit_pages, seo_issues (broken links, missing meta...)
  |
  ├── FetchGSCDataJob        (queue: google-apis)
  |     → seo_keyword_ranks (pozycje, kliknięcia, impressions, CTR)
  |
  ├── FetchGA4DataJob        (queue: google-apis) ← równolegle z GSC
  |     → organic traffic, conversions per page
  |
  ├── AnalyzeKeywordsJob     (queue: ai-processing)
  |     → seo_keywords (klasyfikacja, grupy tematyczne)
  |
  ├── AnalyzeSemanticsJob    (queue: ai-processing) ← równolegle
  |     → seo_recommendations (AI: Claude API / OpenAI)
  |
  ├── GenerateAuditReportJob
  |     → audits.overall_score = avg(component scores)
  |     → seo_analyses.status → 'completed'
  |
  └── NotifyAuditCompleteJob
        → notifications (type: audit_complete)
        → broadcast → bell w UI
```

TimescaleDB — seo_keyword_ranks

Tabela `seo_keyword_ranks` to hypertable TimescaleDB partycjonowany po kolumnie `time` . Umożliwia zapytania time-series: trend pozycji w czasie, rolling average, continuous aggregates. Dane z Google Search

> Console mogą mieć miliony wierszy — standardowy PostgreSQL nie skaluje się dla takich zapytań. TimescaleDB to rozszerzenie PostgreSQL — nie zmienia stacku.

13

UI

Struktura menu — panel admina i klienta

Panel admina — app.clientops.pl

- Dashboard
- Leady + Kanban
- Klienci
- Kontakty
- Projekty
- Oferty
- Dokumenty
- Analizy SEO
- Audyty
- Zgłoszenia (Tickets)
- SLA / Naruszenia
- Inbox / Powiadomienia
- Użytkownicy
- Ustawienia + Audit log

Panel klienta — {slug}.clientops.pl

- Dashboard
- Moje projekty
- Dokumenty
- Analizy SEO
- Audyty
- Zgłoszenia
- SLA — status umowy
- Inbox / Powiadomienia
- Profil

Różnice w widokach

Sekcja	Admin widzi	Klient widzi
Leady	Wszystkie, Kanban, aktywności, konwersja	— (niedostępne)
Oferty	Pełne zarządzanie, wersje dokumentu	— (niedostępne)
Kontakty	Wszystkie per klient, edycja	Własne dane

Sekcja	Admin widzi	Klient widzi
Projekty	Wszyscy klienci, pełne dane, deadline	Tylko swoje, ograniczony widok
Zgłoszenia	Wszystkie, komentarze wewnętrzne, notatki	Tylko swoje, bez notatek wewnętrznych
SLA	Incydenty, konfiguracja polityk, breach history	Status umowy — czy SLA jest spełniane
SEO / Audyty	Pełne dane, uruchamianie analiz, AI rekomendacje	Wyniki i raporty (read-only)
Inbox	Notyfikacje system + wiadomości od klientów + admin↔admin	Wiadomości od admina + notyfikacje systemowe
Audit log	Historia zmian per rekord + widok globalny (super-admin)	— (niedostępne)

ClientOps CRM — Architektura i Model Danych v2.0.0 · Marzec 2026 · 27 tabel · 13 sekcji · Laravel 12 + React 19 + PostgreSQL + TimescaleDB

📂 Dokumentacja ClientOps CRM — powiązane dokumenty

📋 Specyfikacja Techniczna v1.5
clientops-crm-spec.html
Stack, moduły, decyzje techniczne

🏗 Ten dokument
Architektura i Model Danych v2.0.0
ERD, schematy tabel, przepływy, 27 tabel

🎨 Design System — Biblioteki v1.0
clientops-libraries.html · clientops-structure.html
Mapowanie JS, tokeny CSS, komponenty

🛠 Przewodnik Implementacji v1.0
clientops-implementation.html
Kontrolery, migracje, testy, CI/CD